

102.4: Instalação e gerenciamento de pacotes de software

Rafael Obelheiro

OTCC Linux

Instalação de software no Linux

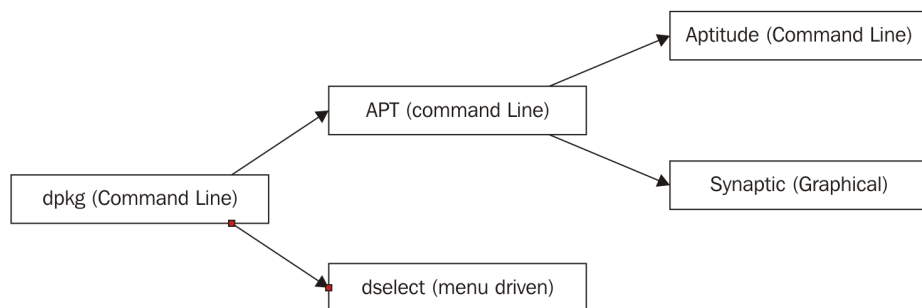
- No passado, a forma mais comum de instalar software no Linux consistia em
 1. Obter o código fonte
 2. Compilar o software
 3. Instalar os binários localmente
 4. Configurar o software
- Muitas vezes, os passos 1–4 precisam ser repetidos para as dependências do software desejado
 - não raro, o problema é descoberto apenas durante a compilação
- É comum surgirem conflitos de dependências
 - o software X usa a versão M da biblioteca `libxpto`, enquanto o software Y usa a versão N, e as versões são incompatíveis
- Atualização pode ser um desafio
 - além do tempo para recompilar, há o risco de quebrar alguma coisa

Sistemas de gerenciamento de pacotes

- Gerenciadores de pacotes foram introduzidos para facilitar o processo de instalação e gerenciamento de software
- Distribuição de binários
 - elimina tempo de compilação
 - frequentemente menores que os fontes
- Automatização das dependências
 - ao instalar um pacote X, todas as suas dependências são instaladas
 - ★ em muitos casos há menos dependências de execução do que de compilação
 - coexistência de múltiplas versões
- Atualização simplificada
 - importante para correção de bugs e vulnerabilidades

Principais gerenciadores de pacotes do Linux

- Os dois principais gerenciadores de pacotes são `dpkg` e `RPM`
 - `dpkg` foi criado pelo Debian, e usa pacotes `.deb`
 - `RPM` foi criado pela Red Hat, e usa pacotes `.rpm`
 - várias distribuições usam um desses dois gerenciadores
 - ★ `dpkg`: Debian, Ubuntu, Mint
 - ★ `RPM`: Red Hat (+ derivados), Fedora, (open)SuSE
- São funcionalmente equivalentes, mas incompatíveis
 - as ferramentas são bem diferentes
 - é inviável misturar pacotes `.deb` e `.rpm` no mesmo sistema
 - ★ existem iniciativas como formatos neutros e conversores de formato, mas nunca vi darem muito certo
- Ambos sistemas dispõem de ferramentas de baixo nível (`dpkg`, `rpm`) e front-ends de mais alto nível (`APT`, `DNF`, ...)



- Ferramenta de baixo nível para gerenciar pacotes .deb
- Pode instalar pacotes locais (já baixados no disco) ou remotos
- Tratamento de dependências não é automatizado
 - `dpkg` verifica se dependências estão satisfeitas, mas não instala pacotes faltantes automaticamente
- Pacotes podem ser obtidos de mídias de instalação ou de repositórios na Internet
 - como Linux é multiplataforma, certificar-se de usar pacotes da arquitetura de máquina correta
 - ★ i386 (x86-32), x86_64, ...

Exemplos (1)

```
### Instala o pacote zoom_amd64.deb
### O mesmo comando atualiza o pacote se já instalado
$ sudo dpkg -i zoom_amd64.deb

### Lista os pacotes instalados
$ dpkg -l [zoom]
$ dpkg -l | grep -i zoom

### Remove o pacote zoom, deixando arqs config modificados
$ sudo dpkg -r zoom

### Remove completamente o pacote zoom ("purge")
$ sudo dpkg -P zoom

### Lista o conteúdo do pacote zoom
$ dpkg -L zoom
```

Exemplos (2)

```
### Identifica o pacote dono de /usr/bin/zoom
$ dpkg -S /usr/bin/zoom

### Lista as dependências do pacote zoom
$ dpkg -s zoom | grep ^Depends

### Lista os pacotes que dependem do pacote zoom
$ sudo dpkg --purge --dry-run zoom

### Mostra informações sobre o pacote zoom
$ dpkg -s zoom          ## se já instalado
$ dpkg -I zoom_amd64.deb ## extrai infos do arquivo
```

Remoção de pacotes

- Existem duas operações de remoção de pacotes, *remove* e *purge*
- *Remove* (`dpkg -r`) não remove arquivos de configuração modificados localmente
 - caso o pacote tenha sido removido acidentalmente, basta reinstalá-lo para restaurar a funcionalidade anterior
- *Purge* (`dpkg -P`) remove todo o conteúdo do pacote, **incluindo** arquivos de configuração modificados localmente
 - se o pacote for reinstalado, as configurações terão de ser refeitas
- É possível fazer *purge* de um pacote que já tenha sido removido
 - isso elimina todos os resquícios do pacote no sistema

Ferramentas de alto nível (APT et al.)

- Ferramentas de alto nível, como APT, facilitam o gerenciamento de pacotes `.deb`
 - simplificam a localização e obtenção de pacotes
 - automatizam o processo de atualização
 - resolvem dependências automaticamente
- Essas ferramentas usam repositórios de pacotes (locais e/ou remotos)
 - cliente deve ser configurado com os repositórios adequados
- Um cache local é usado para encontrar atualizações e dependências
 - deve ser atualizado periodicamente
- Distribuições para desktop geralmente possuem um gerenciador de atualizações
 - automatiza o ciclo `update + upgrade`

Exemplos (1)

Atualiza a lista de pacotes

```
$ sudo apt update
```

Lista pacotes disponíveis/instalados

```
$ apt list
```

```
$ apt list --installed
```

Procura o pacote zoom (no cache)

```
$ apt search zoom
```

```
$ apt-cache search --names-only zoom
```

Mostra informações sobre o pacote zoom

```
$ apt show zoom
```

Instala o pacote zoom

```
$ sudo apt install zoom
```

Exemplos (2)

Remove/expurga o pacote zoom

```
$ sudo apt remove zoom
```

```
$ sudo apt purge zoom
```

Atualiza o pacote zoom/todos os pacotes

```
$ sudo apt upgrade zoom
```

```
$ sudo apt upgrade
```

Lista pacotes [instalados] que dependem de zoom

```
$ apt rdepends [--installed] zoom
```

Remove dependências que não são mais necessárias

```
$ sudo apt autoremove
```

Limpa o cache de pacotes baixados da Internet

(em `/var/cache/apt`)

```
$ sudo apt clean
```

Repositórios de pacotes

- Os repositórios usados pelo APT são definidos em `/etc/apt/sources.list` e `/etc/apt/sources.list.d`
 - no 2o caso, todos os arquivos no diretório
- As entradas têm o seguinte formato:
`deb <url_repo> <release> <componentes>`
 - `deb` especifica pacotes binários (`deb-src` para fontes)
 - `url_repo` é a URL do repositório
 - `release` é o nome da *release*
 - ★ Ubuntu 24.04.3 LTS: `noble`, Debian 13.1: `trixie`
 - ★ há ainda `noble-updates`, `noble-security`, `noble-backports`
 - `componentes` representa os tipos de pacotes no repositório
 - ★ Ubuntu: `main`, `universe`, `restricted`, `multiverse`
- *Releases* representam conjuntos consistentes de pacotes
 - não se deve misturar pacotes de *releases* diferentes (a menos que se tenha certeza de que são compatíveis)

apt-file

- `apt-file`: localiza o pacote que contém um determinado arquivo
 - `dpkg -S` só funciona para pacotes instalados
- Especialmente útil quando algum software reclama da falta de um arquivo
- Requer a instalação do pacote `apt-file`

Atualiza o cache de `apt-file`

```
$ sudo apt-file update
```

Lista os arquivos do pacote `zoom`

```
$ apt-file list zoom
```

Procura pacotes que contenham `libXt.so`

```
$ apt-file search libXt.so
```

Outras ferramentas

- Synaptic: interface gráfica para o APT
- Tasksel: coleções de pacotes que são instalados como “tarefas”
 - geralmente coleções grandes, como um ambiente de desktop inteiro
 - requer o pacote `tasksel`
 - a lista de tarefas disponíveis pode ser obtida com `tasksel --list`
- Flatpak: pacotes que executam de forma isolada em uma *sandbox*
 - geralmente inclui as dependências necessárias, sem preocupação de compatibilidade com os pacotes `.deb` da distribuição
- Snap: contêineres de aplicação desenvolvidos pela Canonical (Ubuntu)
 - a aplicação e todas as suas dependências são encapsuladas dentro de um snap, que executa em uma *sandbox*
- Flatpak e Snap podem ser úteis para executar aplicações complexas incompatíveis com a distribuição
 - o principal custo é o desperdício de disco e memória

Instalação a partir dos fontes (1)

- Em alguns casos, pode ser necessário instalar a partir dos fontes
 - um software indispensável não está disponível nos repositórios
- Os fontes são disponibilizados em `.tar.gz` ou outro formato de compressão
- O procedimento habitual consiste em
 1. Descompactar os fontes
 - ★ sugere-se usar `/usr/local/src` para esse fim
 2. Compilar os fontes

```
$ ./configure
```

```
$ make
```
 3. Instalar os binários

```
$ sudo make install
```
 4. Configurar o software
 - os passos 1 e 2 não requerem privilégios, basta ajustar as permissões

Instalação a partir dos fontes (2)

- É importante documentar como foi feita a compilação (ainda que tenha sido com a receita de bolo tradicional) e qualquer configuração adicional, para facilitar futuras instalações
 - ▶ a documentação não é para os outros, é para você mesmo daqui a 3 meses
 - ▶ convém manter o documento na própria máquina e fazer uma cópia de segurança

Algumas recomendações (1)

- Segurança
 - ▶ pensar antes de adicionar repositórios e baixar pacotes não oficiais
 - ▶ cuidar com avisos de chaves e assinaturas inválidas
 - ★ <https://wiki.debian.org/SecureApt>
 - ▶ evitar instalar software desnecessário
 - ★ bugs em software não instalado são irrelevantes
- Se possível, preferir versões estáveis e distribuições com *long-term support* (LTS)
 - ▶ menos bugs e vulnerabilidades para se preocupar
 - ▶ minimiza atualizações forçadas da distribuição
- Evitar atualizações automáticas não assistidas

Algumas recomendações (2)

- Usar espelhos oficiais da distribuição
 - ▶ melhor desempenho no acesso a repositórios
 - ▶ para atualizações de segurança, melhor usar o repositório principal
 - ▶ com grandes parques de máquinas, pode ser uma boa ideia manter um espelho local dos pacotes do SO
- Atualizar a versão da distribuição via gerenciador de pacotes (`dist-upgrade`) costuma deixar “cracas”